```
  });
});
```

① Creates a key for the S3 object

② Calls the S3 API to upload an object

③ The S3 bucket name is passed in as an environment variable (the bucket will be created later in the chapter).

④ The body is the uploaded stream of data.

⑤ Calls the DynamoDB API to update an object

⑥ Updates the state, version, and raw S3 key

⑦ Updates only when the item exists. Version equals the expected version, and state is one of those allowed.

⑧ Calls the SQS API to publish a message

⑨ Creates the message body containing the image's ID and the desired state

⑩ The queue URL is passed in as an environment variable.

Registers the route with Express

We are using the multiparty module to handle multipart uploads.

The server side is finished. Next, you'll continue to implement the processing part in the Imagery worker. After that, you can deploy the application.

16.3.3 Implementing a fault-tolerant worker to consume SQS messages

The Imagery worker processes images by applying a sepia filter asynchronously. The worker runs through the following steps in an endless loop. It is worth noting that multiple workers can run at the same time:

1. Poll the queue for new messages.
2. Fetch the process data from the database.
3. Download the image from S3.